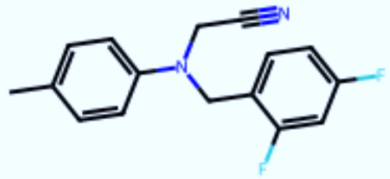
DSA103 Advanced Chemical Data Science

Lecture 16: Data wrangling with loss

Dr. Johannes Schörgenhumer (johannes.schoergenhumer@chem.uzh.ch)

	Smiles	Standard Type	Standard Relation	Standard Value	Standard Units	Assay ChEMBL ID	ROMol
...					g.mL-1	CHEMBL3625335	
	@H]1CC[C@@H]2CN1C(=O)N2OS(=O)(=O)[O-].[Na+]	MIC	'<='	0.120	ug.mL-1	CHEMBL5247824	
							

Discussion of Assignment_session14

Recap: Data wrangling without loss





- Cleaning, transforming, and reshaping safely
- Every record and value is preserved
- Operations are reversible

Data wrangling – “types”

Cleaning:

- Fix typos
- Convert types
- Fill NAs

No data loss

Transformation:

- Renaming columns
- Pivoting
- Merging

No data loss (if careful)

Reduction:

- Filter
- Aggregate
- Drop columns

Data loss

Pivoting and melting

Opposite operations for reshaping data frames

Melt():

- Convert wide to long (column headers into rows)
- Converts Dataframe to a format of identifier and measured variables
- Common use: Data cleaning, preparation for plotting
- Flexible, handling of duplicates supported

Pivot():

- Convert long to wide (a column with multiple values into several columns of their own)
- Common use: Data summarization
- No handling of duplicates (index!)
- pivot_table() tolerates duplicates, but performs aggregation

https://jakevdp.github.io/PythonDataScienceHandbook/03.09-pivot-tables.html
https://pandas.pydata.org/docs/user_guide/reshaping.html

Melt

df3

	first	last	height	weight
0	John	Doe	5.5	130
1	Mary	Bo	6.0	150



```
df3.melt(id_vars=['first', 'last'])
```

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130
3	Mary	Bo	weight	150

Pivot

```
df.pivot(index='foo',
         columns='bar',
         values='baz')
```

df

	foo	bar	baz	zoo
0	one	A	1	x
1	one	B	2	y
2	one	C	3	z
3	two	A	4	q
4	two	B	5	w
5	two	C	6	t

Stacked



bar	A	B	C
foo			
one	1	2	3
two	4	5	6

Record

Activity: Pivot / melt

1) Demo of pivot and melt

2) Discuss the benefit of a long-table vs. a wide table

3) Play through the exercise on the iris dataset

Joining datasets – concat():

Always check shape before and after transformations!

pd.concat():

- Like **np.concatenate()**, also allowing specification of axis (default: rows)
- By default: preservation of indices (**ignore_index=False**)
- Data source can be added by **keys** option
- Tracking duplication: flag **verify_integrity** raises exception
- Joins can specify the type of concatenation (e.g. inner vs. outer)
- **append()** is a method of Pandas objects as a shorthand for concat

```
try:
    pd.concat([x, y], verify_integrity=True)
except ValueError as e:
    print("ValueError:", e)
```

```
ValueError: Indexes have overlapping values: [0, 1]
```

https://pandas.pydata.org/docs/user_guide/merging.html

Joining datasets – merge():

Always check shape before and after transformations!

merge(): Combines two DataFrames based on common columns or indices

Key parameters:

- **left, right**: The two DataFrames to merge
- **on**: Column(s) to join on (must exist in both DataFrames)
- left_on, right_on: Specify different column names to join on if they differ
- **how**: Type of join
  - **'inner'** → only matching rows (default)
  - 'left' → all rows from left + matches from right
  - 'right' → all rows from right + matches from left
  - 'outer' → all rows from both (union)

```
# Merge on canonical_smiles, how="inner" drops all rows which are only contained in one df
merged = pd.merge(act_clean, desc, on="canonical_smiles", how="inner")
```

https://pandas.pydata.org/docs/user_guide/merging.html

Joining datasets – join():

Always check shape before and after transformations!

join(): Combines two DataFrames **by their index** (or by a key column), similar to **merge()**, but with a simpler syntax (called as method!)

Typical use case: Join key = index

Key parameters:

- other: The DataFrame (or list of DataFrames) to join with
- on: Column(s) to join on (only used when joining on a column instead of the index)
- **how**: Type of join — 'left', 'right', 'outer', 'inner' (default **'left'**)

```
left_df = pd.DataFrame(
    {"A": ["A0", "A1", "A2"], "B": ["B0", "B1", "B2"]}, index=["K0", "K1", "K2"]
    )

right_df = pd.DataFrame(
    {"C": ["C0", "C2", "C3"], "D": ["D0", "D2", "D3"]}, index=["K0", "K2", "K3"]
    )

result_df = left_df.join(right_df)
```

https://pandas.pydata.org/docs/user_guide/merging.html

Activity: Merging dataframes

Add to the previous dataframe from the “patients_results_messy.csv” the extended patient data (“patient_list_extended.csv”) and merge with the “patients_info.csv”

Export the cleaned dataset as csv-file.

Data wrangling with loss





- Cleaning, transforming, and reshaping
- Some records and values are lost (intentionally)
- Operations are irreversible

Data wrangling with loss

Why?

- To simplify, summarize, or extract insights
- To reduce noise or redundancy
- To create new features

Examples:

- Filtering
- Aggregation
- Sampling
- Deduplication
- Removing NAs
- Removing Outliers



Operations with data loss: Filtering

Just keeps data points which meet a certain condition:

- Single condition, e.g. comparators or functions:

```
hypertonic_I = df[df["systolic"] > 130]
```

```
str_filter = df[df["notes"].str.contains("re")]
```

- Multiple conditions: Logic operators & (and), | (or), ~(not), conditions wrapped in parantheses

```
two_cond = df[(df["systolic"] > 130) | (df["chol_mg_dl"] > 200)]
```

- Using isin(): Check if values are in a certain list, e.g.

```
filtered_df = df[df.notes.isin(["repeat", "recheck", "lab values missing"])]
```

- Filter by missing values: notna(), isna()

```
filtered_df = df[df.chol_mg_dl.isna()]
```

Operations with data loss: Sampling

Selecting a subset of data:

- Random sampling, e.g.

```
df.sample(n=5, random_state=42)
```

```
df.sample(frac=0.5)
```

- Stratified sample: keeps the same fraction of each group, e.g.

```
df.groupby("gender", group_keys=False)[df.columns].apply(lambda x: x.sample(frac=0.5))
```

- Systematic sampling:

```
df.iloc[::2]
```

Operations with data loss: Deduplication

How to assess:

1) Check for duplicates and quantify: **df.duplicated().sum()**
2) Inspect full-row duplicates: **df[df.duplicated(keep=False)]**
3) Partial duplicates – only check specific columns: **df.duplicated(subset=[“col1”, “col2”])**
4) Partial duplicates - same ID but different attributes, e.g. after an update): inspect by grouping, e.g. by “name”
5) Decide: Keep, update or drop?

Good practice:

- Document why duplicates were dropped
- Keep original dataframe

Operations with data loss: Removing missing values

Dropping rows with missing values: **dropna()**

- By default deletes any row that contains at least one missing value, deleting columns can be specified (**dropna(axis=1)**)
- To introduce a threshold for how many NaN are still acceptable: **thresh=x**
- To specify to only drop rows if all entries are missing: **how="all"**
- A subset of columns can be specified to only drop rows if the missing value is in one of them: **dropna(subset=["columns"])**

Alternatively, fill missing values: **fillna()**

- Fill missing value with zero (or any other number, e.g. mean or median): **df.fillna(0)**
- Missing data can also be filled with previous **df["col"].ffill()** or next value in the column **df["col"].bfill()**

Good practice: document and keep copy of original data!

https://jakevdp.github.io/PythonDataScienceHandbook/03.04-missing-values.html

Activity: Operations with loss

Run different operations on the data of the cleaned and merged data from the patient dataset. Consider:

1) Missing data
2) Outliers
3) Duplicates
4) Filtering (try different conditions)
5) Sampling

…inspect the data before and afterwards (describe, etc.)

…assert which data or if patterns have been lost (visualisation, e.g. as histogram for a feature after outlier removal)